FUNDACIÓN UNIVERSITARIA AGRARIA DE COLOMBIA – UNIAGRARIA
Dirección de Educación Virtual · Ciencia de Datos (Profesional)
Actividad 2 – Hands on: configuración de herramientas
Estudiante: Andres Felipe Amaya Camacho
Docente: Carlos Andres Gomez Vasco

Capítulo 5. Git en entornos distribuidos

Flujos de trabajo distribuidos, cómo contribuir y cómo mantener un proyecto

Con un DVCS son posibles muchas formas de colaborar. Este capítulo describe los flujos más comunes y las buenas prácticas para contribuir a un proyecto y para mantenerlo.

5.1 Flujos de trabajo distribuidos

- **Flujo centralizado:** un único repositorio compartido al que todos hacen push; hay que integrar antes de subir (`git pull` y luego `git push`).
- **Gestor de integración:** cada persona tiene su repositorio público de escritura propio y un responsable integra los cambios en el repositorio oficial. Es el modelo de GitHub (fork + pull request).
- **Dictador y tenientes:** variante para proyectos enormes (como el núcleo de Linux), con jerarquía de revisores.

5.2 Contribuir a un proyecto

Antes de contribuir conviene conocer las reglas del proyecto. Algunas pautas generales:

- Evitar errores de espacios en blanco: `git diff --check`.
- Hacer commits **atómicos**: cada uno con un solo cambio lógico, que se pueda entender y revertir por separado.
- Escribir buenos mensajes: una línea de resumen de unos 50 caracteres, línea en blanco y, si hace falta, un cuerpo que explique el *por qué* del cambio.

```
# Ejemplo de mensaje claro
git commit -m "Corrige el cálculo del promedio en cap. 2" -m "El ejemplo dividía
por la cantidad equivocada de elementos."
```

Para aportar a un proyecto ajeno se suele hacer un *fork* (copia propia en el servidor), clonarlo, crear una rama temática, enviar los cambios a ese fork y abrir una solicitud de integración (pull request). Alternativamente, algunos proyectos reciben parches por correo con `git format-patch` y `git send-email`.

5.3 Mantener un proyecto

Quien mantiene el proyecto revisa y aplica las contribuciones. Algunas herramientas útiles:

```
git switch -c rama-tema main
git apply parche.patch           # aplicar un parche
git am parche.mbox              # aplicar parches de correo
git remote add contribuidor https://github.com/otro/proyecto.git
git fetch contribuidor
git merge contribuidor/rama
git cherry-pick <hash>          # traer un solo commit
git describe main               # nombre legible de una versión
git archive main --format=zip -o release.zip
```

Conclusión del capítulo

La colaboración eficaz depende tanto de las herramientas como de los acuerdos: commits pequeños, mensajes claros y un flujo de integración conocido por todo el equipo.

Fuente: Chacon, S. y Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2